

Практическая работа 22. Рассчитайте пользовательские чаевые

В этом кодовом модуле вы используете код решения из кодового модуля Intro to state in Compose для создания интерактивного калькулятора чаевых, который может автоматически вычислять и округлять сумму чаевых, когда вы вводите сумму счета и процент чаевых. Готовое приложение вы можете увидеть на этом изображении:

d8e768525099378a.png

Пререквизиты Коделаб «Введение в состояние в Compose». Умение добавлять в приложение композитные функции Text и TextField. Знание функции remember(), состояния, поднятия состояния и разницы между композитными функциями с состоянием и без состояния. Что вы узнаете Как добавить кнопку действия в виртуальную клавиатуру. Что такое Switch composable и как его использовать. Добавлять ведущие иконки в текстовые поля. Что вы создадите Приложение Tip Time, которое рассчитывает сумму чаевых на основе введенной пользователем суммы счета и процента чаевых. Что вам понадобится Последняя версия Android Studio Код решения из коделаба Intro to state in Compose

3. Обзор приложения Starter

Этот коделаб начинается с приложения Tip Time из предыдущего коделаба Intro to state in Compose, которое предоставляет пользовательский интерфейс, необходимый для расчета чаевых с фиксированным процентом чаевых. В текстовом поле «Сумма счета» пользователь может ввести стоимость услуги. Приложение рассчитывает и отображает сумму чаевых в текстовом поле Text composable.

Запуск приложения Tip Time Откройте проект Tip Time в Android Studio и запустите приложение на эмуляторе или устройстве. Введите сумму счета. Приложение автоматически рассчитает и отобразит сумму чаевых.

В текущей реализации процент чаевых жестко закодирован на 15%. В этом коделабе вы расширите эту функцию с помощью текстового поля, которое позволит приложению рассчитать пользовательский процент чаевых и округлить сумму чаевых.

Добавьте необходимые строковые ресурсы На вкладке «Проект» нажмите res > values > strings.xml. Между тегами `<resources>` файла strings.xml добавьте эти строковые ресурсы:

```
<string name="how_was_the_service">Tip Percentage</string>
<string name="round_up_tip">Round up tip?</string>
```

Файл strings.xml должен выглядеть так, как показано в этом фрагменте кода, который включает строки из предыдущего коделаба:

strings.xml

```
<resources>
    <string name="app_name">Tip Time</string>
    <string name="calculate_tip">Calculate Tip</string>
    <string name="bill_amount">Bill Amount</string>
    <string name="how_was_the_service">Tip Percentage</string>
    <string name="round_up_tip">Round up tip?</string>
    <string name="tip_amount">Tip Amount: %s</string>
</resources>
```

4. Добавьте текстовое поле для указания процента чаевых

Клиент может захотеть оставить больше или меньше чаевых в зависимости от качества оказанных услуг и по разным другим причинам. Чтобы учесть это, приложение должно позволить пользователю рассчитать индивидуальный размер чаевых. В этом разделе вы добавляете текстовое поле, в котором пользователь может ввести процент чаевых, как показано на этом изображении:

В вашем приложении уже есть композитное текстовое поле Bill Amount, которое представляет собой композитную функцию `EditNumberField()` без состояния. В предыдущем коделабе вы перенесли состояние `amountInput` из составной функции `EditNumberField()` в составную функцию `TipTimeLayout()`, что сделало составную функцию `EditNumberField()` нестационарной.

Чтобы добавить текстовое поле, можно повторно использовать тот же компонент `EditNumberField()`, но с другой меткой. Для этого необходимо передать метку в качестве параметра, а не жестко закодировать ее внутри функции `EditNumberField()` `composable`.

Сделайте композитную функцию `EditNumberField()` многоразовой:

В файле `MainActivity.kt` в параметрах составной функции `EditNumberField()` добавьте ресурс `label string` типа `Int`:

```
@Composable
fun EditNumberField(
    label: Int,
    value: String,
    onValueChanged: (String) -> Unit,
    modifier: Modifier = Modifier
)
```

Примечание: Лучшей практикой является предоставление параметра `Modifier` по умолчанию для всех композитных функций, что увеличивает возможность повторного использования. Вы

должны добавить его в качестве первого необязательного параметра после всех обязательных параметров.

В теле функции замените жестко закодированный строковый идентификатор ресурса на параметр `label`:

```
@Composable
fun EditNumberField(
    //...
) {
    TextField(
        //...
        label = { Text(stringResource(label)) },
        //...
    )
}
```

Чтобы указать, что параметр `label` должен быть ссылкой на строковый ресурс, аннотируйте параметр функции с помощью аннотации `@StringRes`:

```
@Composable
fun EditNumberField(
    @StringRes label: Int,
    value: String,
    onValueChanged: (String) -> Unit,
    modifier: Modifier = Modifier
)
```

Примечание: Примером ссылки на строковый ресурс является `R.string.bill_amount`.

Импортируйте следующее:

```
import androidx.annotation.StringRes
```

Примечание: Аннотация `@StringRes` - это безопасный для типов способ использования строковых ресурсов. Она указывает, что передаваемое целое число является строковым ресурсом из файла `values/strings.xml`. Эти аннотации полезны для разработчиков, которые работают над вашим кодом, и для инструментов проверки кода, таких как `lint` в `Android Studio`. Подробнее о `lint` вы узнаете в одном из следующих кодовых уроков.

В вызове функции `EditNumberField()` композитной функции `TipTimeLayout()` установите параметр `label` в строковый ресурс `R.string.bill_amount`:

```
EditNumberField(
    label = R.string.bill_amount,
    value = amountInput,
```

```
onValueChanged = { amountInput = it },  
modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()  
)
```

В панели предварительного просмотра не должно быть никаких визуальных изменений.

В составной функции `TipTimeLayout()` после вызова функции `EditNumberField()` добавьте еще одно текстовое поле для пользовательского процента чаевых. Выполните вызов составной функции `EditNumberField()` с такими параметрами:

```
EditNumberField(  
    label = R.string.how_was_the_service,  
    value = "",  
    onValueChanged = { },  
    modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()  
)
```

Это добавляет еще одно текстовое поле для пользовательского процента чаевых.

В предварительном просмотре приложения теперь отображается текстовое поле `Tip Percentage`, как показано на этом изображении:

В верхней части композитной функции `TipTimeLayout()` добавьте свойство `var tipInput` для переменной состояния добавленного текстового поля. Используйте `mutableStateOf(«») для инициализации переменной и окружите вызов функцией remember:`

```
var tipInput by remember { mutableStateOf("") }
```

В новом вызове функции `EditNumberField()` установите значение именованного параметра в переменную `tipInput`, а затем обновите переменную `tipInput` в лямбда-выражении `onValueChanged`:

```
EditNumberField(  
    label = R.string.how_was_the_service,  
    value = tipInput,  
    onValueChanged = { tipInput = it },  
    modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()  
)
```

В функции `TipTimeLayout()` после определения переменной `tipInput`. Определите `val` с именем `tipPercent`, который преобразует переменную `tipInput` к типу `Double`. Используйте оператор `Elvis` и верните 0, если значение равно `null`. Это значение может быть нулевым, если текстовое поле пустое.

```
val tipPercent = tipInput.toDoubleOrNull() ?: 0.0
```

В функции `TipTimeLayout()` обновите вызов функции `calculateTip()` и передайте переменную `tipPercent` в качестве второго параметра:

```
val tip = calculateTip(amount, tipPercent)
```

Код функции `TipTimeLayout()` должен выглядеть так, как показано в этом фрагменте:

```
@Composable
fun TipTimeLayout() {
    var amountInput by remember { mutableStateOf("") }
    var tipInput by remember { mutableStateOf("") }
    val amount = amountInput.toDoubleOrNull() ?: 0.0
    val tipPercent = tipInput.toDoubleOrNull() ?: 0.0

    val tip = calculateTip(amount, tipPercent)
    Column(
        modifier = Modifier.padding(40.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = stringResource(R.string.calculate_tip),
            modifier = Modifier
                .padding(bottom = 16.dp)
                .align(alignment = Alignment.Start)
        )
        EditNumberField(
            label = R.string.bill_amount,
            value = amountInput,
            onValueChanged = { amountInput = it },
            modifier = Modifier
                .padding(bottom = 32.dp)
                .fillMaxWidth()
        )
        EditNumberField(
            label = R.string.how_was_the_service,
            value = tipInput,
            onValueChanged = { tipInput = it },
            modifier = Modifier
                .padding(bottom = 32.dp)
                .fillMaxWidth()
        )
        Text(
            text = stringResource(R.string.tip_amount, tip),
            style = MaterialTheme.typography.displaySmall
        )
    }
}
```

```

        Spacer(modifier = Modifier.height(150.dp))
    }
}

```

Запустите приложение на эмуляторе или устройстве, а затем введите сумму счета и процент чаевых. Правильно ли приложение рассчитывает сумму чаевых?

5. Настройка кнопки действия

В предыдущем коделабе вы изучили, как использовать класс `KeyboardOptions` для установки типа клавиатуры. В этом разделе вы узнаете, как установить кнопку действия клавиатуры с помощью того же `KeyboardOptions`. Кнопка действия клавиатуры - это кнопка в конце клавиатуры. Некоторые примеры вы можете увидеть в этой таблице:

Property	Action button on the keyboard
<code>ImeAction.Search</code> . Used when the user wants to execute a search.	
<code>ImeAction.Send</code> . Used when the user wants to send the text in the input field.	
<code>ImeAction.Go</code> . Used when the user wants to navigate to the target of the text in the input.	

В этом задании вы устанавливаете две различные кнопки действия для текстовых полей:

Кнопка действия «Далее» для текстового поля «Сумма счета», которая указывает, что пользователь закончил ввод и хочет перейти к следующему текстовому полю. Кнопка действия «Готово» для текстового поля «Процент чаевых», которая указывает, что пользователь закончил ввод. Примеры клавиатур с этими кнопками действий можно увидеть на этих изображениях:

Добавьте опции клавиатуры:

В вызове функции `TextField()` функции `EditNumberField()` передайте конструктору `KeyboardOptions` именованный аргумент `imeAction`, установленный на значение `ImeAction.Next`. Используйте функцию `KeyboardOptions.Default.copy()`, чтобы убедиться, что вы используете другие опции по умолчанию.

```

import androidx.compose.ui.text.input.ImeAction

@Composable
fun EditNumberField(

```

```

    //...
  ) {
    TextField(
      //...
      keyboardOptions = KeyboardOptions.Default.copy(
        keyboardType = KeyboardType.Number,
        imeAction = ImeAction.Next
      )
    )
  }

```

Запустите приложение на эмуляторе или устройстве. На клавиатуре теперь отображается кнопка «Следующее действие», как показано на этом изображении:

Обратите внимание, что на клавиатуре отображается одна и та же кнопка действия Next (Далее), когда выбрано текстовое поле Tip Percentage (Процент наконечника). Однако вам нужны две разные кнопки действий для текстовых полей. Вскоре вы решите эту проблему.

Изучите функцию `EditNumberField()`. Параметр `keyboardOptions` в функции `TextField()` жестко закодирован. Чтобы создать разные кнопки действий для текстовых полей, нужно передать в качестве аргумента объект `KeyboardOptions`, что вы и сделаете в следующем шаге.

```

// No need to copy, just examine the code.
fun EditNumberField(
    @StringRes label: Int,
    value: String,
    onValueChanged: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    TextField(
        //...
        keyboardOptions = KeyboardOptions.Default.copy(
            keyboardType = KeyboardType.Number,
            imeAction = ImeAction.Next
        )
    )
}

```

В определении функции `EditNumberField()` добавьте параметр `keyboardOptions` типа `KeyboardOptions`. В теле функции присвойте его именованному параметру `keyboardOptions` функции `TextField()`:

```

@Composable
fun EditNumberField(
    @StringRes label: Int,
    keyboardOptions: KeyboardOptions,
    // ...
){

```

```
        TextField(  
            //...  
            keyboardOptions = keyboardOptions  
        )  
    }
```

В функции `TipTimeLayout()` обновите первый вызов функции `EditNumberField()`, передайте именованный параметр `keyboardOptions` для текстового поля `Bill Amount`:

```
, // ... ))
```

Во втором вызове функции `EditNumberField()` измените значение `imeAction` текстового поля `Tip Percentage` на `ImeAction.Done`. Ваша функция должна выглядеть так, как показано в этом фрагменте кода:

```
, // ... ))
```

Запустите приложение. Оно отобразит кнопки действий `Next` и `Done`, как показано на этих изображениях:

Введите любую сумму счета и нажмите кнопку `Далее`, а затем введите любой процент чаевых и нажмите кнопку `Готово`. После этого клавиатура закрывается.

6. Добавьте переключатель

Переключатель включает или выключает состояние одного элемента.

В тумблере есть два состояния, которые позволяют пользователю выбирать между двумя вариантами. Тумблер состоит из дорожки, большого пальца и дополнительной иконки, как показано на этих изображениях:

Переключатель - это элемент управления выбором, который можно использовать для принятия решений или объявления предпочтений, например, настроек, как показано на этом изображении:

Пользователь может провести большим пальцем вперед-назад, чтобы выбрать нужную опцию, или просто нажать на переключатель для переключения. Еще один пример переключения можно увидеть в этом GIF, где параметр «Визуальные параметры» переключается в режим «Темный»:

Чтобы узнать больше о переключателях, обратитесь к документации по переключателям.

Вы используете составной переключатель Switch, чтобы пользователь мог выбрать, округлять ли чаевые до ближайшего целого числа, как показано на этом изображении:

Добавьте строку для композитов Text и Switch:

После функции EditNumberField() добавьте композитную функцию RoundTheTipRow(), а затем передайте модификатор по умолчанию в качестве аргументов, аналогичных функции EditNumberField():

```
@Composable
fun RoundTheTipRow(modifier: Modifier = Modifier) {
}
```

Реализуйте функцию RoundTheTipRow(), добавьте компоновку Row со следующим модификатором, чтобы установить ширину дочерних элементов максимальной на экране, выровнять по центру и обеспечить размер 48dp:

```
Row(
    modifier = modifier
        .fillMaxWidth()
        .size(48.dp),
    verticalAlignment = Alignment.CenterVertically
) {
}
```

Импортируйте следующее:

```
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.size
```

В блоке лямбда компоновщика Row layout добавьте компоновщик Text, который использует строковый ресурс R.string.round_up_tip для отображения строки Round up tip?

```
Text(text = stringResource(R.string.round_up_tip))
```

После составного элемента Text добавьте составной элемент Switch, передайте ему именованный параметр checked, установив его на roundUp, и именованный параметр onCheckedChange, установив его на onRoundUpChanged.

```
Switch(  
    checked = roundUp,  
    onCheckedChange = onRoundUpChanged,  
)
```

В этой таблице содержится информация об этих параметрах, которые являются теми же параметрами, которые вы определили для функции `RoundTheTipRow()`:

Parameter	Description
checked	Whether the switch is checked. This is the state of the Switch composable.
onCheckedChange	The callback to be called when the switch is clicked.

Import the following:

```
import androidx.compose.material3.Switch
```

В функцию `RoundTheTipRow()` добавьте параметр `roundUp` типа `Boolean` и лямбда-функцию `onRoundUpChanged`, которая принимает булево значение и ничего не возвращает:

```
@Composable  
fun RoundTheTipRow(  
    roundUp: Boolean,  
    onRoundUpChanged: (Boolean) -> Unit,  
    modifier: Modifier = Modifier  
)
```

Это поднимает состояние переключателя.

В составном переключателе добавьте этот модификатор, чтобы выровнять составной переключатель по краю экрана:

```
Switch(  
    modifier = modifier  
        .fillMaxWidth()  
        .wrapContentWidth(Alignment.End),  
    //...  
)
```

Импортируйте следующее:

```
import androidx.compose.foundation.layout.wrapContentWidth
```

В функции `TipTimeLayout()` добавьте переменную `var` для состояния компонента `Switch`. Создайте переменную с именем `roundUp` и установите ее в `mutableStateOf()` с начальным значением `false`. Окружите вызов символами `{ }`.

```
fun TipTimeLayout() {
    //...
    var roundUp by remember { mutableStateOf(false) }

    //...
    Column(
        ...
    ) {
        //...
    }
}
```

Это переменная для состояния `Switch` composable, и `false` будет состоянием по умолчанию.

В блоке `Column` функции `TipTimeLayout()` после текстового поля `Tip Percentage`. Вызовите функцию `RoundTheTipRow()` со следующими аргументами: именованный параметр `roundUp`, установленный на значение `roundUp`, и именованный параметр `onRoundUpChanged`, установленный на обратный вызов лямбды, который обновляет значение `roundUp`:

```
@Composable
fun TipTimeLayout() {
    //...

    Column(
        ...
    ) {
        Text(
            ...
        )
        Spacer(...)
        EditNumberField(
            ...
        )
        EditNumberField(
            ...
        )
        RoundTheTipRow(
            roundUp = roundUp,
            onRoundUpChanged = { roundUp = it },
            modifier = Modifier.padding(bottom = 32.dp)
        )
        Text(
            ...
        )
    }
}
```

На экране появится строка «Округлять чаевые?»

Запустите приложение. Приложение отобразит тумблер «Округлять чаевые?».

Введите сумму счета и процент чаевых, а затем выберите тумблер «Округлять чаевые?». Сумма чаевых не округляется, потому что вам все равно нужно обновить функцию `calculateTip()`, что вы и сделаете в следующем разделе.

Обновите функцию `calculateTip()`, чтобы округлить чаевые

Измените функцию `calculateTip()`, чтобы она принимала булеву переменную для округления чаевых до ближайшего целого числа:

Для округления чаевых функция `calculateTip()` должна знать состояние переключателя, которое является булевой переменной. В функцию `calculateTip()` добавьте параметр `roundUp` типа `Boolean`:

```
private fun calculateTip(  
    amount: Double,  
    tipPercent: Double = 15.0,  
    roundUp: Boolean  
): String {  
    //...  
}
```

В функции `calculateTip()` перед оператором `return` добавьте условие `if()`, которое проверяет значение `roundUp`. Если значение `roundUp` истинно, определите переменную `tip` и установите ее в функцию `kotlin.math.ceil()`, а затем передайте функцию `tip` в качестве аргумента:

```
if (roundUp) {  
    tip = kotlin.math.ceil(tip)  
}
```

Предупреждение. Android Studio выдает ошибку - `val cannot be reassigned error`, необходимо изменить переменную `tip` на переменную `var`:

```
var tip = tipPercent / 100 * amount
```

Примечание: Функция `kotlin.math.ceil(x)` округляет заданное значение целого числа в большую сторону. Например, она округляет 10,65 до 11,00. Эта функция может принимать число `Double` или `Float`.

The completed `calculateTip()` function should look like this code snippet:

```
private fun calculateTip(amount: Double, tipPercent: Double = 15.0, roundUp: Boolean): String {
    var tip = tipPercent / 100 * amount
    if (roundUp) {
        tip = kotlin.math.ceil(tip)
    }
    return NumberFormat.getCurrencyInstance().format(tip)
}
```

В функции `TipTimeLayout()` обновите вызов функции `calculateTip()`, а затем передайте параметр `roundUp`:

```
val tip = calculateTip(amount, tipPercent, roundUp)
```

Запустите приложение. Теперь оно округляет сумму чаевых, как вы можете видеть на этих изображениях:

7. Добавьте поддержку альбомной ориентации

Устройства Android выпускаются в различных форм-факторах - телефоны, планшеты, раскладушки и устройства ChromeOS - и имеют широкий диапазон размеров экрана. Ваше приложение должно поддерживать как портретную, так и альбомную ориентацию.

Протестируйте приложение в ландшафтном режиме, включите автоповорот.

Поверните эмулятор или устройство влево, и вы заметите, что не можете увидеть сумму чаевых. Чтобы решить эту проблему, вам понадобится вертикальная полоса прокрутки, которая поможет вам прокрутить экран приложения.

Добавьте к модификатору `.verticalScroll(rememberScrollState())`, чтобы включить вертикальную прокрутку колонки. Функция `rememberScrollState()` создает и автоматически запоминает состояние прокрутки.

```
@Composable
fun TipTimeLayout() {
    // ...
    Column(
        modifier = Modifier
            .padding(40.dp)
            .verticalScroll(rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally,
```

```
        verticalArrangement = Arrangement.Center
    ) {
        //...
    }
}
```

Импортируйте следующее:

```
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
```

Запустите приложение снова. Попробуйте прокрутку в ландшафтном режиме!

8. Добавьте ведущий значок в текстовые поля (необязательно)

Значки могут сделать текстовое поле более визуально привлекательным и предоставить дополнительную информацию о текстовом поле. Значки могут использоваться для передачи информации о назначении текстового поля, например, о том, какой тип данных ожидается или какой тип ввода требуется. Например, значок телефона рядом с текстовым полем может означать, что пользователь должен ввести номер телефона.

Значки могут использоваться для руководства вводом данных пользователем, предоставляя визуальные подсказки о том, что ожидается. Например, значок календаря рядом с текстовым полем может указывать на то, что пользователь должен ввести дату.

Ниже приведен пример текстового поля со значком поиска, указывающим на необходимость ввода поискового запроса.

Добавьте в составной `EditNumberField()` еще один параметр `leadingIcon` типа `Int`. Аннотируйте его с помощью `@DrawableRes`.

```
@Composable
fun EditNumberField(
    @StringRes label: Int,
    @DrawableRes leadingIcon: Int,
    keyboardOptions: KeyboardOptions,
    value: String,
    onValueChanged: (String) -> Unit,
    modifier: Modifier = Modifier
)
```

Импортируйте следующее:

```
import androidx.annotation.DrawableRes
import androidx.compose.material3.Icon
```

Добавьте ведущий значок в текстовое поле. `LeadingIcon` принимает составной элемент, вы передадите в него следующий составной элемент `Icon`.

```
TextField(
    value = value,
    leadingIcon = { Icon(painter = painterResource(id = leadingIcon), null) },
    //...
)
```

Передайте ведущую иконку в текстовые поля. Для удобства иконки уже присутствуют в стартовом коде.

```
EditNumberField(
    label = R.string.bill_amount,
    leadingIcon = R.drawable.money,
    // Other arguments
)
EditNumberField(
    label = R.string.how_was_the_service,
    leadingIcon = R.drawable.percent,
    // Other arguments
)
```

Запустите приложение.